Absolute Difference

Instead of looking for exact equality with floats, you should use RSpec's be_within matcher:

11-matchers-included-in-rspec-expectations/01/primitive_matchers.rb
```
expect(0.1 + 0.2).to be_within(0.0001).of(0.3)
```

The value we've passed to be_within here is the *delta*, or absolute difference in either direction. This particular expectation passes as long as the value is between 0.2999 and 0.3001 (which it is, of course).

Relative Difference

Equally useful is the percent_of method, where you give a *relative* difference instead:

11-matchers-included-in-rspec-expectations/01/primitive_matchers.rb
```
town_population = 1237
expect(town_population).to be_within(25).percent_of(1000)
```

We've used integers for this example to show that range-related matchers aren't just for floating-point numbers. All the matchers in this section work just fine for both types of numbers. Integer approximation comes in handy when you're testing things that have slightly different values each time, such as memory consumption.

A single be_within matcher supports both absolute and relative values, based on which method you chain off of it. This style is called a *fluent interface*, and it helps you write expectations that read naturally: "expect [actual] to be within [delta] of [expected]."[3]

As you will see throughout the rest of this chapter, many of the other built-in matchers support this same kind of fluent interface. It is easy to add it to your custom matchers as well, as you will see in *Adding a Fluent Interface*, on page 222.

Ranges

Sometimes, it's a better fit to express your expected values in terms of a *range*, rather than a target value and delta. For these situations, you can use the be_between matcher:

11-matchers-included-in-rspec-expectations/01/primitive_matchers.rb
```
expect(town_population).to be_between(750, 1250)
```

3. https://martinfowler.com/bliki/FluentInterface.html